Agenda

Deadlocks

Modeling resource use with graphs

Avoiding deadlock

- Ignore it
- Detection and recovery
- Dynamic avoidance
- Prevention

Introduction to Deadlocks

Formal definition : *A set of processes is deadlocked if every process is waiting for an event that only another process can provide*

Typically involves a circular dependency on a resource

-> P1 is waiting on P2 who waits for P3 who is waiting for P1

Deadlock: Example

Idea: Transferring money from A to B and from B to A simultaneously

```
void *Transfer(void *args){
   //argument passing removed to increase readability
   //...
   pthread_mutex_lock(&fromAcct->lock);
   pthread_mutex_lock(&toAcct->lock);

   fromAcct->balance -= amt;
   toAcct->balance += amt;

   pthread_mutex_unlock(&fromAcct->lock);
   pthread_mutex_unlock(&toAcct->lock);
   return NULL;
}
```

Deadlock: example

Thread 0

```
Transfer(…) {
    //acctA is fromAcct
    //acctB is toAcct
    pthread_mutex_lock(&acctA->lock);
        Thread 0 gets here
    pthread_mutex_lock(&acctB->lock);
```

Thread 1

```
Transfer(…) {
    //acctB is fromAcct
    //acctA is toAcct
    pthread_mutex_lock(&acctB->lock);
        Thread 1 gets here
    pthread_mutex_lock(&acctA->lock);
```

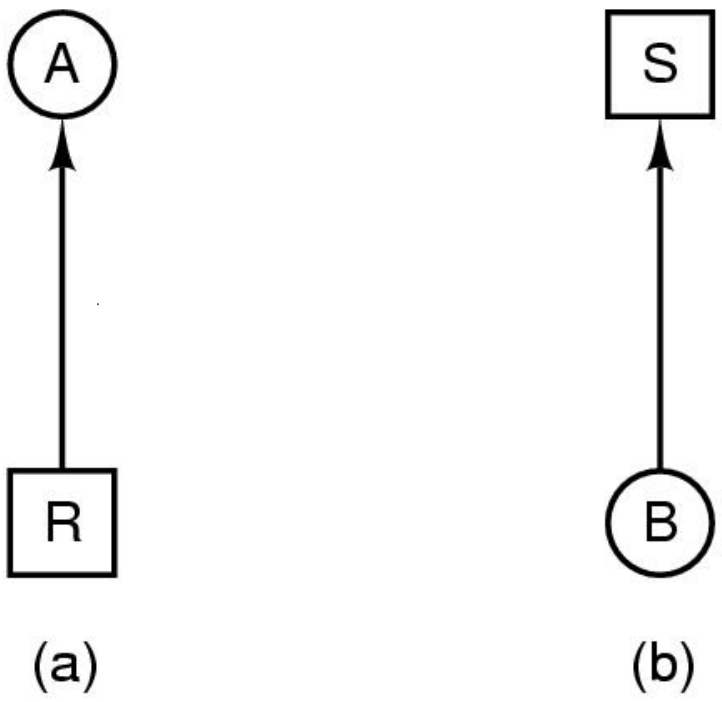
Figure 1. An example of deadlock

Suppose that Threads 0 and 1 are executing concurrently and represent users A and B, respectively. Now consider the situation in which A and B want to transfer money to each other: A wants to transfer 20 dollars to B, while B wants to transfer 40 to A.

From *Dive into systems*

Deadlock: Visualization

Circles represent processes/threads
Squares represent resources



A **holds** resource R

B **requests** resource R

Resource Allocation Graph (RAG)

If our graph has a cycle, then we have deadlock

Deadlock: Visualization

Thread 0

```
Transfer(…) {
    //acctA is fromAcct
    //acctB is toAcct
    pthread_mutex_lock(&acctA->lock);
        Thread 0 gets here
    pthread_mutex_lock(&acctB->lock);
```

Thread 1

```
Transfer(…) {
    //acctB is fromAcct
    //acctA is toAcct
    pthread_mutex_lock(&acctB->lock);
        Thread 1 gets here
    pthread_mutex_lock(&acctA->lock);
```

Four Conditions for Deadlock

1. Mutual exclusion condition

   Each resource is either assigned to a process or is available

2. Hold and wait condition

   Process holding resources can request additional resources

3. No preemption condition

   Previously granted resources cannot be taken away

4. Circular wait condition

   Must be a circular chain of 2 or more processes

   Each is waiting for resource held by next member of the chain

Breaking any of these conditions allows us to avoid/break deadlock

Deadlock: Fixed

```
void *Transfer(void *args){
   //argument passing removed to increase readability
   //...

   pthread_mutex_lock(&fromAcct->lock);
   fromAcct->balance -= amt;
   pthread_mutex_unlock(&fromAcct->lock);

   pthread_mutex_lock(&toAcct->lock);
   toAcct->balance += amt;
   pthread_mutex_unlock(&toAcct->lock);
   return NULL;
}
```

This solution removes the possibility of a circular wait

The processes no longer try to hold more than one resource at once.

Exercise: Detecting deadlock

Visualize the following requests for resources using a graph

1. A requests R
2. B requests S
3. C requests T
4. A requests S
5. B requestsT
6. C requests R

Does deadlock occur?

Exercise: Detecting deadlock

Visualize the following requests for resources using a RAG

1. A requests R
2. C requests T
3. A requests S
4. C requests R
5. A releases R
6. A releases S

Does deadlock occur?

Exercise: Detecting Deadlocks

Suppose there is only one instance of each resource

Example 1: Is this a deadlock?

- P1 has R2 and R3, and is requesting R1
- P2 has R4 and is requesting R3
- P3 has R1 and is requesting R4

Exercise: Detecting Deadlocks

Suppose there is only one instance of each resource

Example 2: Is this a deadlock?

- P1 has R2, and is requesting R1 and R3
- P2 has R4 and is requesting R3
- P3 has R1 and is requesting R4

Dealing with Deadlocks

1. Ignore it
   A better strategy than it sounds

2. Detection and recovery
   Resource Allocation Graphs

3. Dynamic avoidance
   Careful resource allocation

4. Prevention
   Negating one of the four necessary conditions

1. Ignore it: The Ostrich Algorithm

Pretend there is no problem

Reasonable if

- deadlocks occur very rarely
- cost of prevention is high

UNIX and Windows take this approach

It is a trade off between

- convenience/speed
- correctness

2. Deadlock Detection and Recovery

Goal: How can OS detect when there is a deadlock?

OS should keep track of

- Current resource allocation (who has what)
- Current pending requests (who is waiting for what)
- Construct a resource allocation graph (RAG) and check for cycles

What can the OS do once a deadlock is detected?

- Kill a low priority process
- Revoke currently allocated resources (if that’s possible)
- Inform the users or the administrator

Multiple Resource Instances

Our previous examples all had a single instance of each resource. What if we have multiple instances?

- Example: Is this a deadlock?
  - Suppose there are 2 instances of A and 3 of B
  - Process P currently has 1 instance of A, and is requesting 1 instance of A and 3 instances of B
  - Process Q currently has 1 instance of B, and is requesting 1 instance of A and 1 instance of B

Solution: Construct a table…

Example: Modeling multiple resources

Approach: Build tables to model owned and requested resources

Example: Is this a deadlock?

- Suppose there are 2 instances of X and 3 of Y
- Process P currently has 1 instance of X, and is requesting 1 instance of X and 3 instances of Y
- Process Q currently has 1 instance of Y, and is requesting 1 instance of X and 1 instance of Y

Example: Modeling multiple resources

Approach: Check the requests in each row against the currently available resources

If there is a sequence for which all process requests can be fulfilled, there is no deadlock

$$A = (1, 2)$$

$$E = (2, 3)$$

$$C = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$R = \begin{bmatrix} 1 & 3 \\ 1 & 1 \end{bmatrix}$$

Multiple Resource Case

- There are n process P_1 .... P_n and m resources E_1 .... E_m
- We maintain the following data structures
  - Existence vector E: E[j] is the total number of instances of resource E_j in existence
  - Current allocation matrix C: C[i,j] is the number of instances of resource E_j currently held by process P_i
  - Current request matrix R: R[i,j] is the number of instances of resource E_j currently being requested by process P_i
  - Availability vector A: A[j] is the number of instances of resource E_j currently free.

Data Structures for Multiple Resource Instances

Resources in existence
$(E_1, E_2, E_3, \ldots, E_m)$

Resources available
$(A_1, A_2, A_3, \ldots, A_m)$

Current allocation matrix

$$\begin{bmatrix} C_{11} & C_{12} & C_{13} & \cdots & C_{1m} \\ C_{21} & C_{22} & C_{23} & \cdots & C_{2m} \\ \vdots & \vdots & \vdots & & \vdots \\ C_{n1} & C_{n2} & C_{n3} & \cdots & C_{nm} \end{bmatrix}$$

Row n is current allocation to process n

Request matrix

$$\begin{bmatrix} R_{11} & R_{12} & R_{13} & \cdots & R_{1m} \\ R_{21} & R_{22} & R_{23} & \cdots & R_{2m} \\ \vdots & \vdots & \vdots & & \vdots \\ R_{n1} & R_{n2} & R_{n3} & \cdots & R_{nm} \end{bmatrix}$$

Row 2 is what process 2 needs

$$\sum_{i=1}^{n} C_{ij} + A_j = E_j$$

Why?

Example

```
L = {} /* List of processes that can be unblocked */
```

```
Allocation Matrix     Request Matrix
    | R1  R2  R3          | R1  R2  R3
----------------      ----------------
P1 | 1   1   1        P1 | 3   2   1
P2 | 2   1   2        P2 | 2   2   1
P3 | 1   1   0        P3 | 0   0   1
P4 | 1   1   1        P4 | 1   1   1
```

Which process has a satisfiable request?

```
A = (0, 0, 1) /* available resources */
```

Request by process i can be satisfied if the row R[i] is smaller than or equal to the vector A

After First Iteration

```
L = { P3 }

 Allocation Matrix    Request Matrix
     | R1   R2   R3        | R1   R2   R3
----------------     ----------------
P1 | 1    1    1      P1 | 3    2    1
P2 | 2    1    2      P2 | 2    2    1
P3 |                  P3 |
P4 | 1    1    1      P4 | 1    1    1

A = (1, 1, 1)
```

Which process has a satisfiable request?

Note: P3's allocation has been added to A

After Second Iteration

```
L = { P3, P4}
```

```
 Allocation Matrix   Request Matrix
    | R1  R2  R3        | R1  R2  R3
----------------    ----------------
P1 | 1   1   1      P1 | 3   2   1
P2 | 2   1   2      P2 | 2   2   1
P3 |                P3 |

P4 |               P4 |
```

```
A = (2, 2, 2).
```

Which process has a satisfiable request?

Recovery from Deadlock

Recovery through preemption

- Take a resource from some other process
- Depends on nature of the resource

Recovery through rollback

- Checkpoint a process periodically
- Use this saved state
- Restart the process if it is found deadlocked

Recovery through killing processes

- kill one of the processes in the deadlock cycle
- the other processes get its resources
- choose process that can be rerun from the beginning

3. Deadlock Avoidance

Motivation: “Is there an algorithm that can always avoid deadlock by conservatively making the right/safe choice all the time?”

Answer is a qualified “yes, if we have certain information in advance”

Main ideas are based on the concept of safe states.

Safe state

An allocation state is **safe** if there is an ordering of processes, a **safe sequence**, such that:

- a first process can finish for sure: there are enough unallocated resources to satisfy all of its claims.
- if the first process releases its currently held resources, the next process can finish for sure (even if it asks all its claim), and so on.

The state is safe because OS can definitely avoid deadlock by blocking any new processes, or any new requests, until all the current processes have finished in the safe order.

Safe State

	Has	Max
A	3	9
B	2	4
C	2	7

Free: 3

Show that providing 2 resources to B corresponds to a safe state

Unsafe State

	Has	Max
A	3	9
B	2	4
C	2	7

Free: 3

Show that providing 1 resource to A corresponds to an unsafe state

Banker's Algorithm

Suppose there are n process P_1, ... P_n and m resources E_1 .... E_m

Suppose every process has declared in advance, its **claim**, the maximum number of resources it will ever need

To avoid deadlocks, OS maintains the **allocation state**

- Current allocation matrix C: C[i,j] is the number of instances of resource R_j currently held by process P_i
- Claims matrix R: R[i,j] is the maximum number of instances of resource E_j that process P_i will ever request
- Availability vector A: A[j] is the number of instances of resources E_j currently free.

Example: Banker's Algorithm for Multiple Resources

Process	Tape drives	Plotters	Scanners	CD ROMs
A	3	0	1	1
B	0	1	0	0
C	1	1	1	0
D	1	1	0	1
E	0	0	0	0

Resources assigned

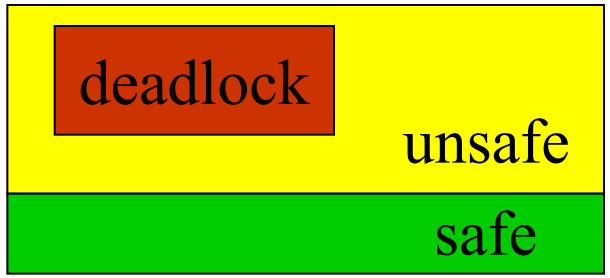
Process	Tape drives	Plotters	Scanners	CD ROMs
A	1	1	0	0
B	0	1	1	2
C	3	1	0	0
D	0	0	1	0
E	2	1	1	0

Resources still needed

E = (6342)
P = (5322)
A = (1020)

Banker's algorithm

- Suppose process P_i requests in R[i] such that each R[i] <= A and R[i]+C[i] <= E[i]
- Consider the state resulting from granting this request (i.e. by adding R[i] to C[i] and subtracting R[i] from A). Check if the new state is a safe state. If so, grant the request, else deny it.
- It ensures that allocation state is always safe



The Banker's Algorithm is conservative:
it cautiously avoids entering an unsafe state even if this unsafe state has no deadlock.

Checking Safety

- How do we check if an allocation state is safe?
  - Current allocation matrix C
  - Remaining claims matrix R
  - Availability vector A

- Same as running the **deadlock detection** algorithm when assuming that every process has requested maximum possible resources

Example: Banker's Algorithm for Multiple Resources

Process	Tape drives	Plotters	Scanners	CD ROMs
A	3	0	1	1
B	0	1	0	0
C	1	1	1	0
D	1	1	0	1
E	0	0	0	0

Resources assigned

Process	Tape drives	Plotters	Scanners	CD ROMs
A	1	1	0	0
B	0	1	1	2
C	3	1	0	0
D	0	0	1	0
E	2	1	1	0

Resources still needed

E = (6342)
P = (5322)
A = (1020)

Which process can complete first?

Example: Banker's Algorithm for Multiple Resources

Process	Tape drives	Plotters	Scanners	CD ROMs
A	3	0	1	1
B	0	1	0	0
C	1	1	1	0
D	1	1	0	1
E	0	0	0	0

Resources assigned

Process	Tape drives	Plotters	Scanners	CD ROMs
A	1	1	0	0
B	0	1	1	2
C	3	1	0	0
D	0	0	1	0
E	2	1	1	0

Resources still needed

E = (6342)
P = (5322)
A = (1020)

Which process can complete?

Example: Banker's Algorithm for Multiple Resources

Process	Tape drives	Plotters	Scanners	CD ROMs
A	3	0	1	1
B	0	1	0	0
C	1	1	1	0
D	1	1	0	1
E	0	0	0	0

Resources assigned

Process	Tape drives	Plotters	Scanners	CD ROMs
A	1	1	0	0
B	0	1	1	2
C	3	1	0	0
D	0	0	1	0
E	2	1	1	0

Resources still needed

E = (6342)
P = (5322)
A = (1020)

Which process can complete?

Example: Banker's Algorithm for Multiple Resources

Process	Tape drives	Plotters	Scanners	CD ROMs
A	3	0	1	1
B	0	1	0	0
C	1	1	1	0
D	1	1	0	1
E	0	0	0	0

Resources assigned

Process	Tape drives	Plotters	Scanners	CD ROMs
A	1	1	0	0
B	0	1	1	2
C	3	1	0	0
D	0	0	1	0
E	2	1	1	0

Resources still needed

E = (6342)
P = (5322)
A = (1020)

Which process can complete?

Digression: Banker’s algorithm

In practice, this is rarely used

Most of the time, it is impossible for a process to know what it will need in advance.

In theory, there is no difference between theory and practice. In practice, there is.

4. Deadlock Prevention

Goal: Can OS ensure that deadlocks never happen?

There are four necessary conditions for deadlocks to occur (what are they?)

Can these conditions be negated ?

Attacking the Mutual Exclusion

Some devices (such as printer) can be spooled

- Only the printer daemon uses printer resource
- thus deadlock for printer eliminated

Principle

- avoid assigning resource when not absolutely necessary
- as few processes as possible actually claim the resource

Not all devices can be spooled

Attacking the Hold and Wait

Require processes to request resources before starting

- a process never has to wait for what it needs

Problems

- may not know required resources at start of run
- also ties up resources other processes could be using

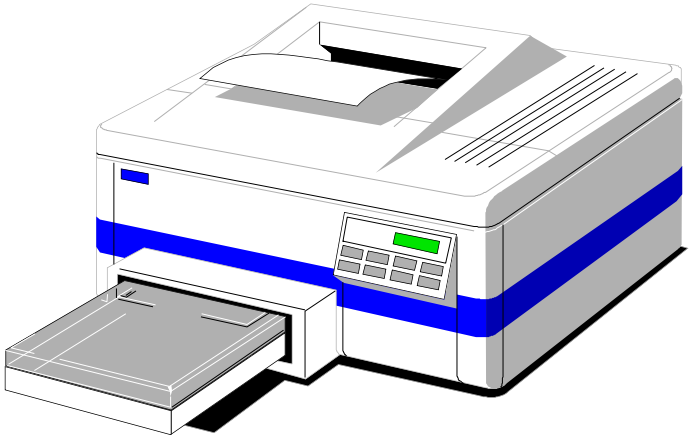
Variation:

- process must first give up all current resources
- then request all needed at once

Attacking the No Preemption

- We can preempt **virtualized** devices that checkpoint and/or buffer input.
  - A virtualized device has a layer between the device and the user that hides implementation details for the user.

- Example: Save printer output to disk so it can be printed once the printer becomes available
  - Requires sufficient disk space
    - We usually have enough disk space



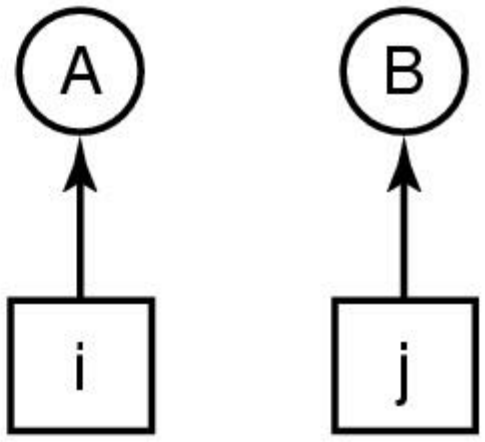
Avoiding the circular wait

Hierarchical Allocation

- Resources are grouped into levels (i.e. prioritize resources)
- A process may only request resources at levels higher than any resource currently held by that process.
- Resources may be released in any order.
- Examples:
  - You can request a scanner then a printer, but if you have the printer, you cannot request the scanner.
  - The dining philosopher's solution required the lower numbered fork to always be picked up first

Attacking the Circular Wait

1. Imagesetter
2. Scanner
3. Plotter
4. Tape drive
5. CD Rom drive



Idea: Processes allowed to request resources in numerical order, e.g. A can request the scanner and then the CD ROM. But not the plotter and then the scanner.

Cycles are impossible. To see why, notice that A can request j but B cannot request i.

Properties of Hierarchical Allocation

The resulting allocation graph never has cycles. Why?

When all requests are at the same level, this method is equivalent to one-shot allocation.

- A process has to request all the resources in one shot

Resources at lower levels are blocked for longer periods, but those at higher levels are shared well.

Summary of approaches for deadlock prevention

Condition	Approach
Mutual exclusion	Spool everything
Hold and wait	Request all resources initially
No preemption	Take resources away
Circular wait	Order resources numerically